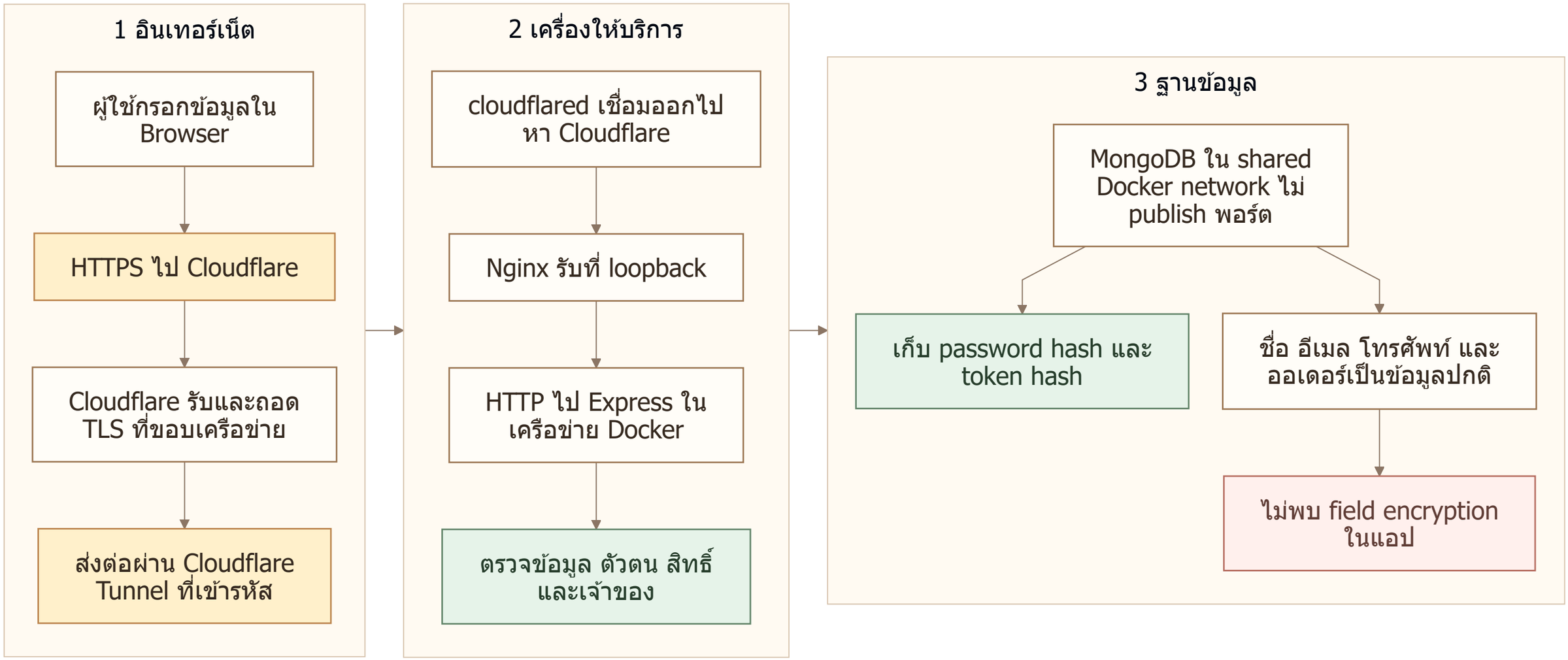
01 ข้อมูลเดินทางผ่านแนวป้องกันใดบ้าง

อ่านจากซ้ายไปขวา: การเข้ารหัสระหว่างส่ง การตรวจคำขอ และการเก็บข้อมูลเป็นคนละชั้นของการป้องกัน



ข้อมูลที่ส่ง: ข้อมูลบัญชีและออเดอร์ผ่าน HTTPS | ข้อมูลที่เก็บ: hash สำหรับรหัสลับ แต่ PII ยังเป็นค่าปกติ

1. HTTPS ป้องกันการดักอ่านระหว่าง Browser กับ Cloudflare; Cloudflare เป็นจุดจบ TLS และจึงประมวลผลคำขอได้ จากนั้นใช้ Tunnel ที่เข้ารหัสถึง cloudflared

2. ตาม production compose: เว็บผูกกับ loopback ส่วน API และ MongoDB ไม่ publish พอร์ต แต่ใช้ shared Docker network ไม่ได้แยกจากทุก container การส่งต่อ Nginx → API เป็น HTTP ภายใน

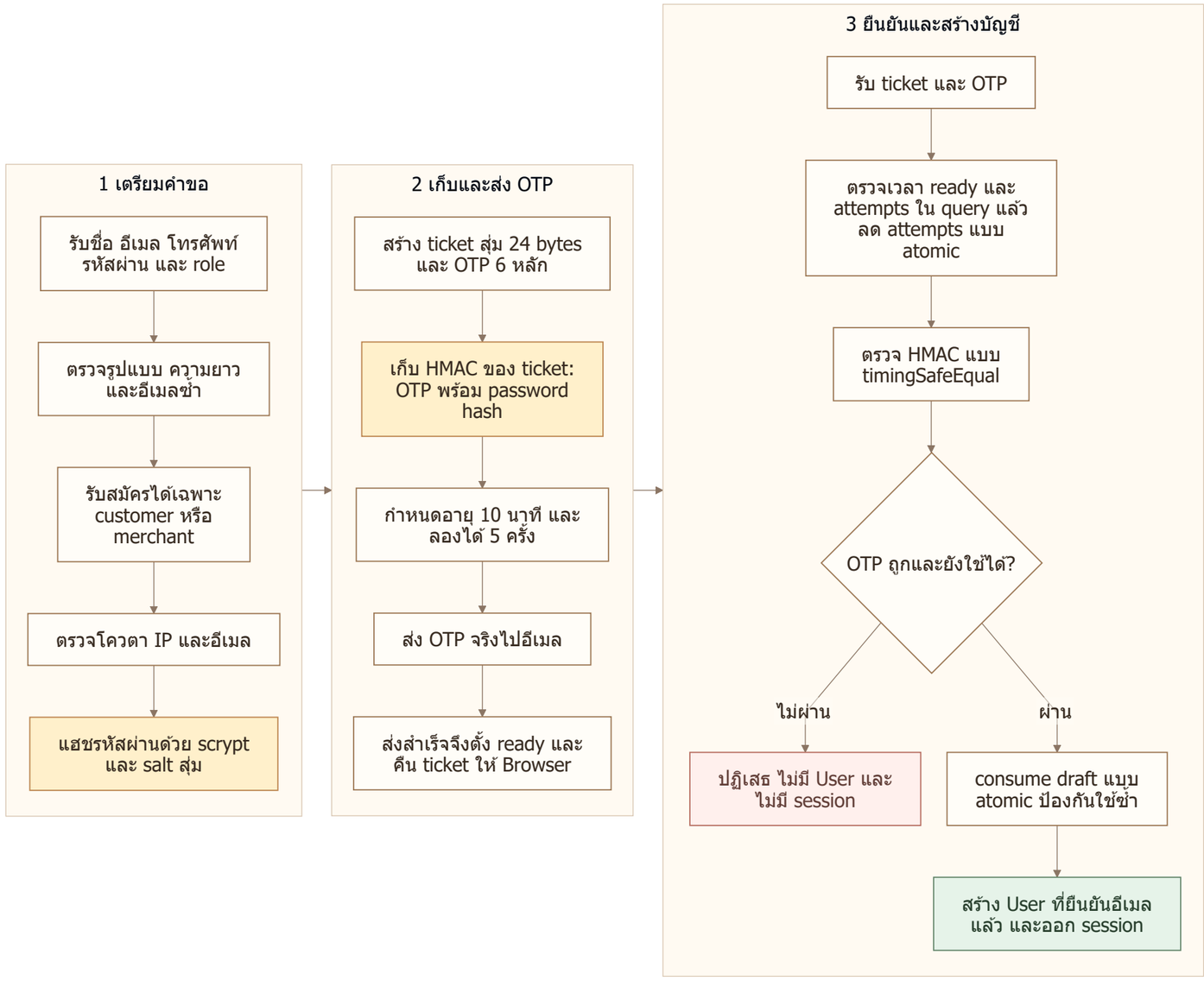
3. ไม่พบ field-level encryption ของข้อมูลส่วนตัว หรือ TLS ของการเชื่อม MongoDB ในค่าตั้งต้น และยังไม่ได้ตรวจการเข้ารหัสดิสก์หรือสถานะเครื่องจริง

4. CORS ตรวจ Origin ของ Browser ตาม allowlist; clients ที่ไม่มี Origin ยังเข้า API ได้และต้องผ่านการตรวจตัวตน/สิทธิ์เช่นเดิม

Source: server.js · cors.js · db.js · compose.yaml · nginx.conf · tunnel.compose.yaml | รายละเอียดอ้างอิงใน README

02 สมัครบัญชี: ยืนยันอีเมลก่อนสร้างผู้ใช้

รหัสผ่านถูกแฮชก่อนเก็บ draft; OTP จริงส่งทางอีเมลและมีอายุจำกัด



รหัสผ่าน: scrypt hash | OTP: HMAC-SHA256 | ticket: สุ่ม 24 bytes | OTP หมดอายุ: 10 นาที

1. รหัสผ่าน 8-128 ตัว ต้องมีตัวอักษรและตัวเลข; self-registration สร้าง admin หรือ staff ไม่ได้ ร้านใหม่อยู่สถานะ pending และปิดรับออเดอร์

2. ส่ง OTP ได้ 5 ครั้ง/อีเมล/ชั่วโมง และ 20 ครั้ง/IP/ชั่วโมง; verify ได้ 100 ครั้ง/IP/ชั่วโมง; ส่งซ้ำรอ 60 วินาทีและทำให้ OTP เก่าใช้ไม่ได้

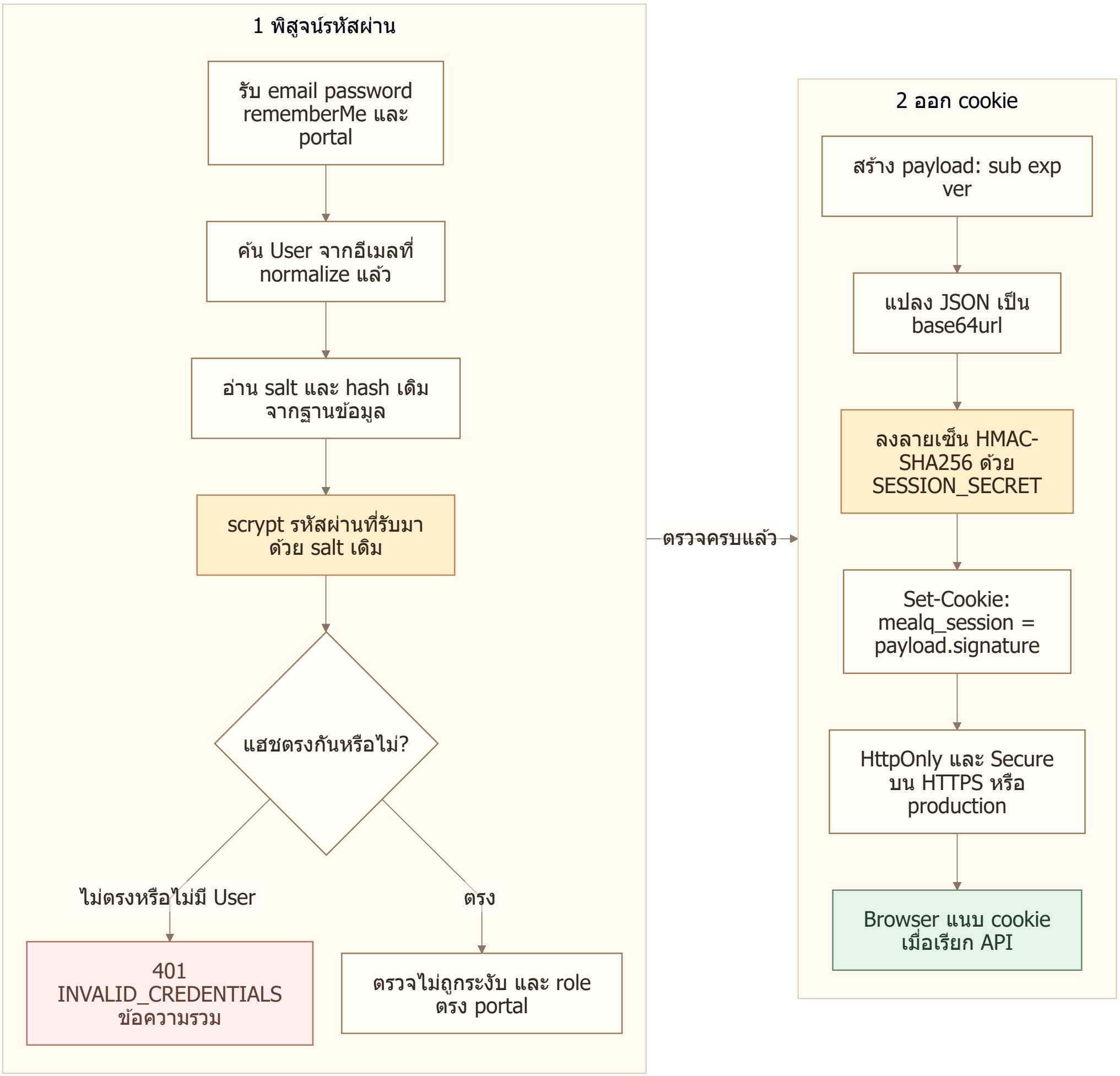
3. PendingRegistration เก็บ password_hash, otp_hash, ticket และข้อมูล draft; ไม่เก็บรหัสผ่านหรือ OTP ตรง ๆ การเทียบ HMAC ใช้ timingSafeEqual

4. TTL ลบ draft หลัง 24 ชั่วโมงเป็นงานทำความสะอาด ส่วนอายุ OTP 10 นาทีตรวจใน query จึงไม่ต้องรอ TTL worker; Frontend เก็บเฉพาะ challenge metadata ใน sessionStorage

Source: authController.js · registration-otp.js · auth-validation.js · password.js · PendingRegistration.js · AuthRateLimit.js · registration-challenge.js | รายละเอียดอ้างอิงใน README

03 เข้าสู่ระบบ: ตรวจ password hash แล้วออก cookie

แฮชรหัสผ่านช่วยลดความเสียหายเมื่อฐานข้อมูลรั่ว ส่วนลายเซ็น session ช่วยตรวจว่าข้อมูลถูกแก้ไขหรือไม่



> ไม่คืน password_hash/reset_token_hash ผ่าน userView | ไม่เก็บ session token ใน localStorage | cookie เป็น bearer credential

1. ตอนสร้างรหัสผ่านใช้ salt สุ่มใหม่ 16 bytes และ scrypt ได้ derived key 64 bytes เก็บเป็น scrypt$salt$hash; การตรวจใช้ timingSafeEqual แทนการเทียบข้อความทั่วไป

2. Session เป็น custom signed token ไม่ใช่ JWT: payload มี user ID, เวลาหมดอายุ และ session_version เท่านั้น base64url อ่านกลับได้ ลายเซ็นไม่ได้เข้ารหัสข้อมูลให้เป็นความลับ

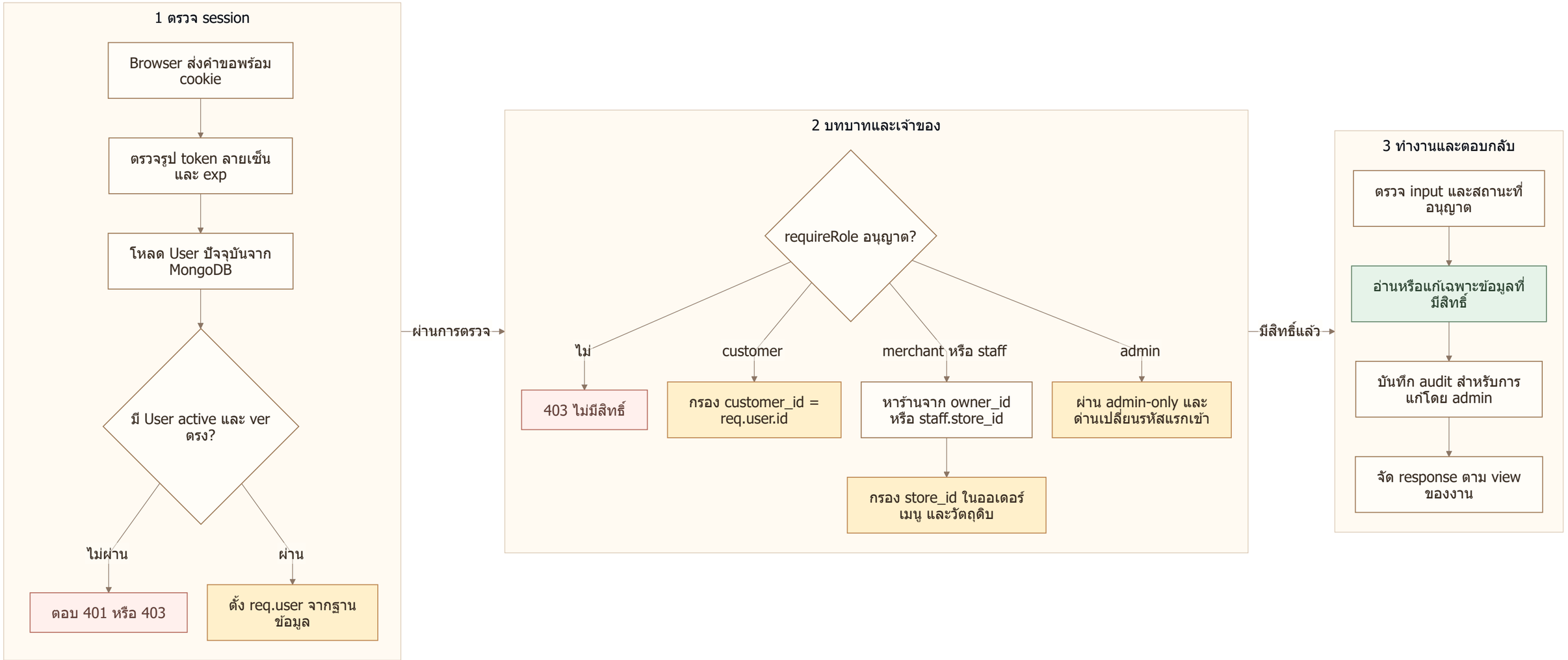
3. จำฉันไว้: token และ cookie อยู่ 30 วัน; ไม่เลือก: token อยู่ 24 ชั่วโมง แต่ cookie ไม่มี maxAge เป็น browser-session cookie ค่า SameSite เป็น None บน HTTPS และ Lax กรณีอื่น

4. HttpOnly ลดการอ่าน cookie ผ่าน JavaScript แต่ไม่กัน XSS ทุกผลกระทบ; ไม่พบ rate limit สำหรับ local login และ logout ลบ cookie เครื่องนั้นโดยไม่ revoke สำเนา token

Source: authController.js · password.js · session.js · user-view.js · api-client.js | รายละเอียดอ้างอิงใน README

04 ทุก API ส่วนตัว: ตรวจตัวตน บทบาท และเจ้าของข้อมูล

การรู้ ID ของออเดอร์หรือร้านไม่เพียงพอ Controller ต้องกรองข้อมูลตามบัญชีที่เซิร์ฟเวอร์ตรวจแล้ว



HTTP 401 = ยังยืนยันตัวตนไม่ได้ | 403 = บัญชีหรือบทบาทไม่อนุญาต | ID จาก URL ต้องผ่าน ownership filter

1. requireAuth ตรวจ session_version กับ User ทุก protected HTTP request จึงตัดสิทธิ์ token เก่าได้หลังเปลี่ยนรหัส อีเมล บทบาท หรือระงับบัญชี; role อ้างอิง DB ไม่เชื่อ role ที่ Browser ส่ง

2. customer ดู/ยกเลิก/รีวิวเฉพาะออเดอร์ตนเอง; merchant ใช้ร้านที่ตนเป็นเจ้าของ; staff ใช้ store_id ที่บัญชีได้รับมอบหมาย โดย staff มีสิทธิ์ปฏิบัติงานร้าน ไม่ใช่สิทธิ์อ่านอย่างเดียว

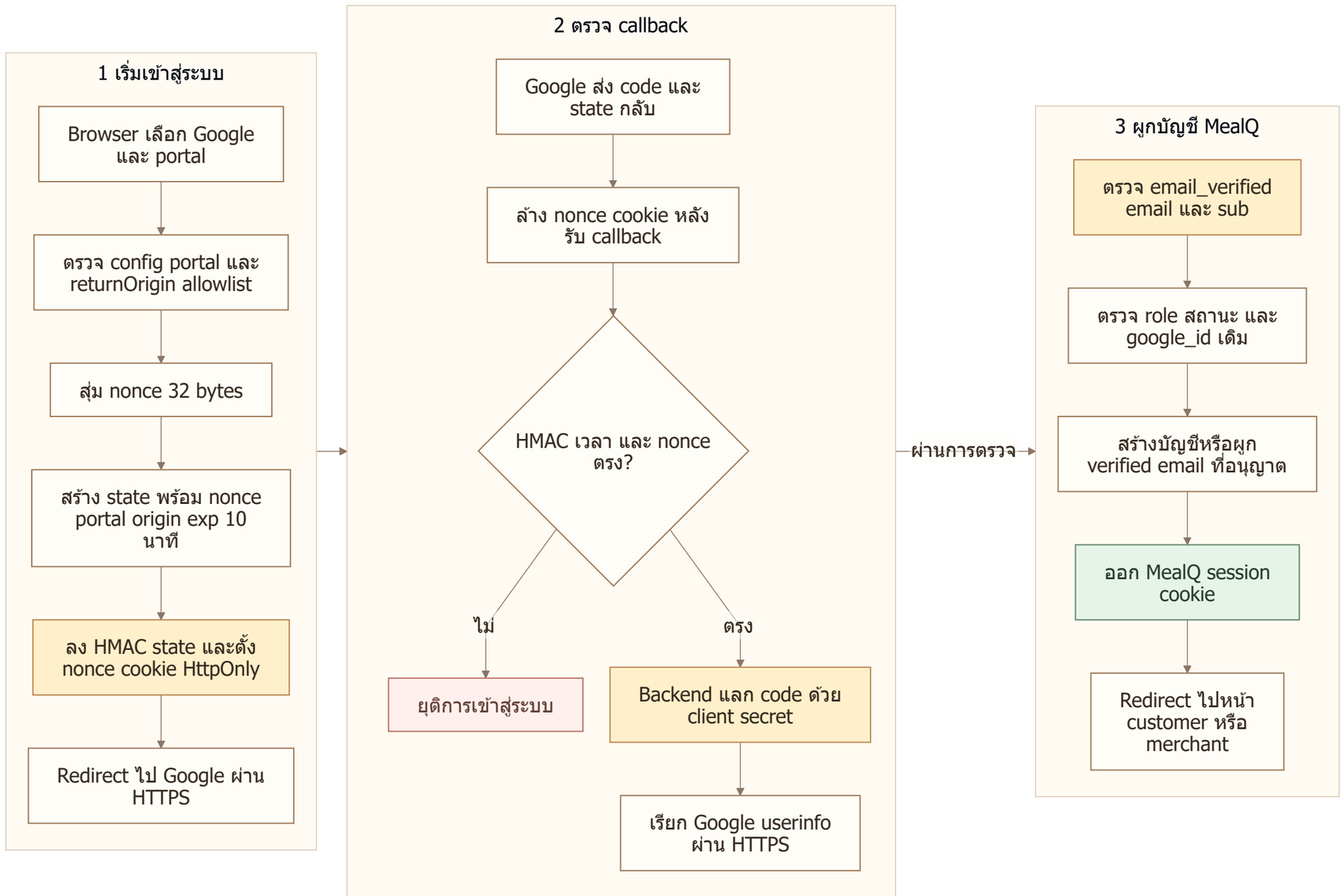
3. admin routes ใช้ role admin และบัญชีที่ must_change_password ต้องเปลี่ยนรหัสก่อนงานสำคัญ; การปรับสิทธิ์ ระงับ และการจัดการโดย admin มี AdminActivity

4. การกรองเจ้าของเกิดในแต่ละ controller ไม่ใช่แค่ซ่อนเมนูหน้าเว็บ และไม่ใช่ MongoDB row-level policy; ต้องตรวจ endpoint ใหม่ทุกครั้งที่เพิ่มฟังก์ชัน

Source: auth.js · customerRoutes.js · orderRoutes.js · merchantRoutes.js · adminRoutes.js · merchant-store.js · orderController.js · merchantController.js · adminController.js | รายละเอียดอ้างอิงใน README

05 Google Login: ไม่ส่งรหัสผ่าน Google ให้ MealQ

state ที่ลงลายเซ็นและ nonce cookie ผูก callback กับ Browser ที่เริ่มคำขอ เพื่อลดการสวม callback ข้ามคำขอ



Browser → Google: Google password | Google → Backend: code/userinfo | Backend → Browser: MealQ cookie

1. ผู้ใช้ป้อนรหัสผ่านที่ Google; MealQ ได้ authorization code และข้อมูลผู้ใช้ที่อนุญาต Backend ใช้ access token ระหว่าง callback ไม่พบการบันทึก token นี้ใน User หรือคืนให้ Frontend

2. GOOGLE_CLIENT_SECRET อยู่ Backend และใช้แลก code กับ Google ส่วน client ID เป็นข้อมูลระบุแอปที่เปิดเผยได้ ไม่ใช่รหัสลับ

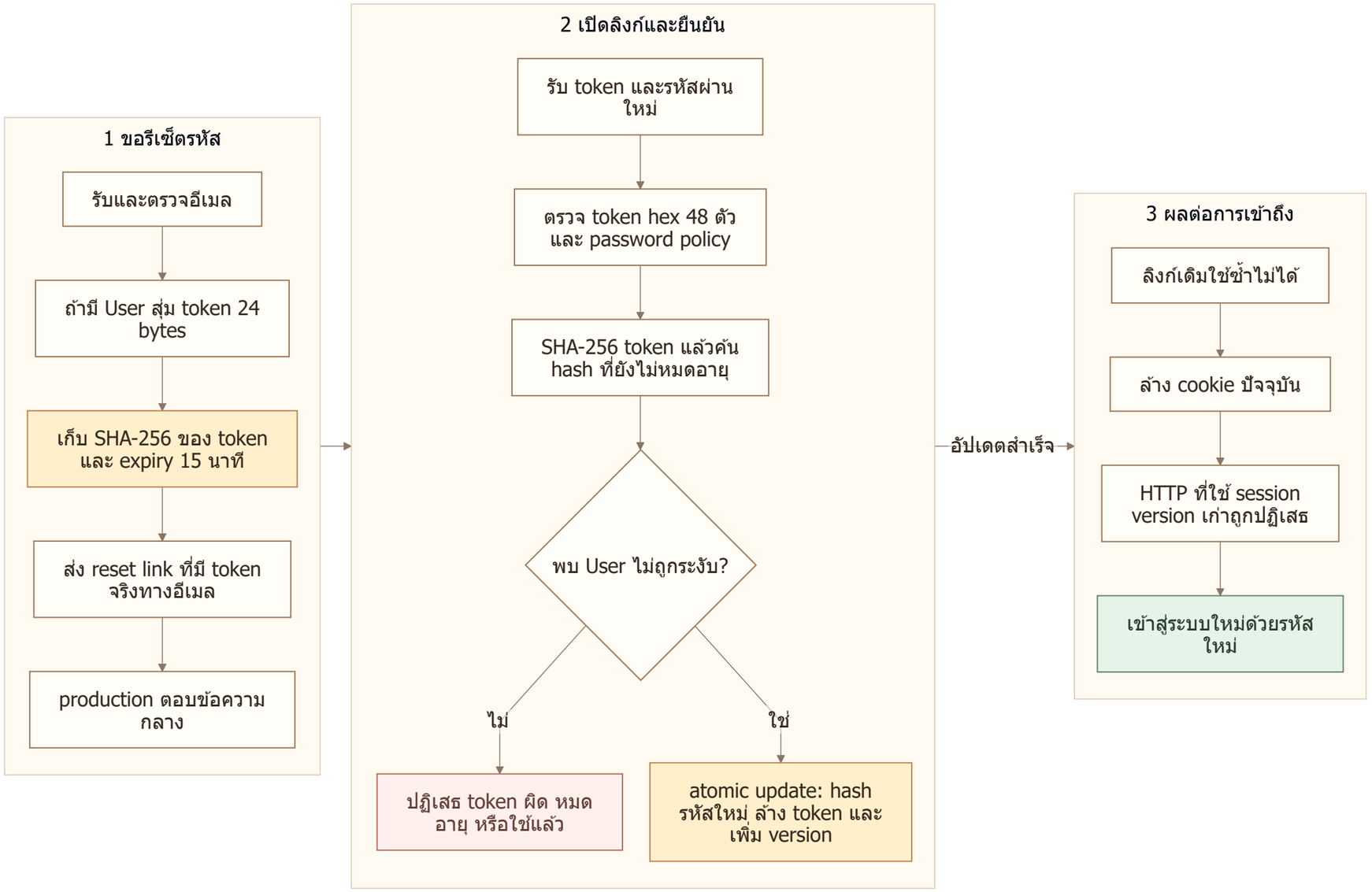
3. state ใช้ HMAC-SHA256 ตรวจการแก้ไข + nonce cookie อายุ 10 นาที; cookie นี้เป็น SameSite=Lax และ Path เฉพาะ Google auth ต่างจาก session cookie หลัก

4. ห้ามเรียก flow นี้ว่า MealQ มี MFA หรือ PKCE: ไม่พบการบังคับ MFA ในแอปหรือ PKCE ใน flow ปัจจุบัน; บัญชีเดิมที่มี google_id ต้องตรงกับ sub

Source: googleAuthController.js · cors.js · session.js · api.js | รายละเอียดอ้างอิงใน README

06 ลืมและเปลี่ยนรหัสผ่าน: token ใช้ครั้งเดียวและยกเลิก session เก่า

ลิงก์ในอีเมลเป็นสิทธิ์ชั่วคราว ผู้ที่ครอบครองลิงก์ก่อนหมดอายุสามารถตั้งรหัสใหม่ได้



reset token: สุ่ม 24 bytes = hex 48 ตัว | hash: SHA-256 | อายุ: 15 นาที | password ใหม่: scrypt + salt ใหม่

1. ฐานข้อมูลเก็บ reset_token_hash ไม่เก็บ token จริง แต่ token อยู่ในลิงก์อีเมลและ URL จึงต้องดูแลกล่องเมล ประวัติ Browser และ log URL ไม่ให้รั่ว

2. reset ใช้ conditional update ตรวจ hash และ expiry แล้วล้าง token พร้อมเปลี่ยนรหัส จึงลดการใช้ซ้ำ; production ไม่คืน token แต่ development อาจคืน devResetToken

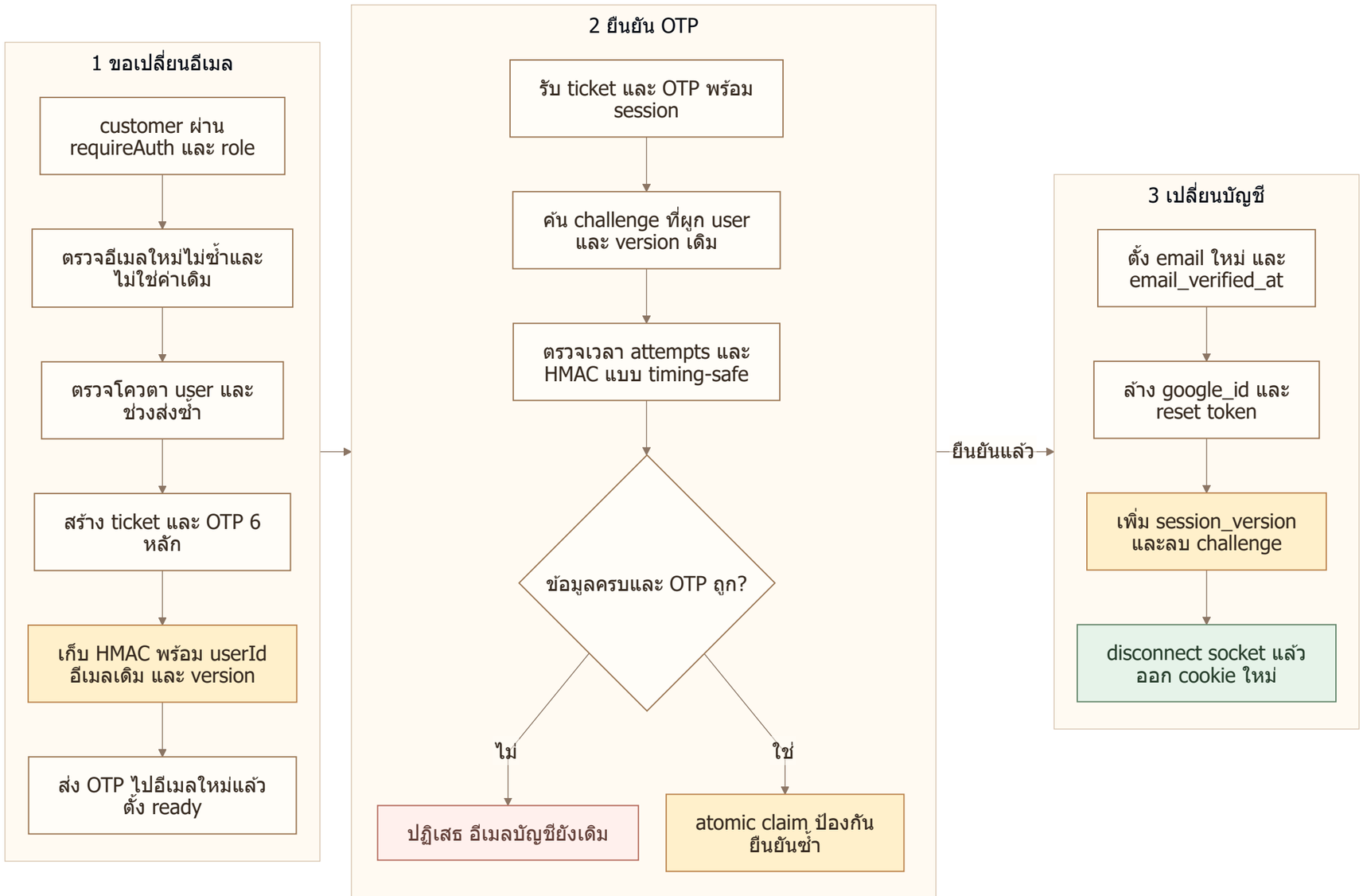
3. เปลี่ยนรหัสจากบัญชีที่ login อยู่ต้องตรวจ current password ก่อน แล้วออก cookie version ใหม่ให้ Browser ปัจจุบัน พร้อม disconnect Socket.IO ของบัญชี

4. ไม่พบ rate limit เฉพาะ forgot/reset; resetPassword เพิ่ม version แต่ยังไม่เรียก disconnectSockets ต่างจาก changePassword จึงควรเสริมการตัด realtime connection เก่าทันที

Source: authController.js · password.js · session.js · authRoutes.js | รายละเอียดอ้างอิงใน README

07 เปลี่ยนอีเมล: ใช้ session ปัจจุบันและ OTP ของอีเมลใหม่

ผูก challenge กับ user ID อีเมลเดิม และ session version เพื่อไม่ให้คำขอค้างถูกนำมาใช้หลังบัญชีเปลี่ยน



บันทึก OTP เป็น HMAC | ticket ผูกบัญชีและ version | เปลี่ยนอีเมลสำเร็จจึงเปลี่ยน User และ session

1. อายุ OTP 10 นาที ลองได้ 5 ครั้ง; ส่ง OTP สูงสุด 5 ครั้ง/บัญชีในหน้าต่าง 1 ชั่วโมง และเว้น 60 วินาที มี rate limiter เพิ่มอีกชั้น 10 คำขอส่งและ 50 คำขอยืนยัน/user/ชั่วโมง; TTL ทำความสะอาด 24 ชั่วโมง

2. แก้ profile ทั่วไปไม่ยอมเปลี่ยนอีเมลโดยตรง ต้องผ่าน flow นี้ ข้อมูล challenge ไม่สามารถนำไปใช้กับ user อื่นหรือ version ที่เปลี่ยนไปแล้ว

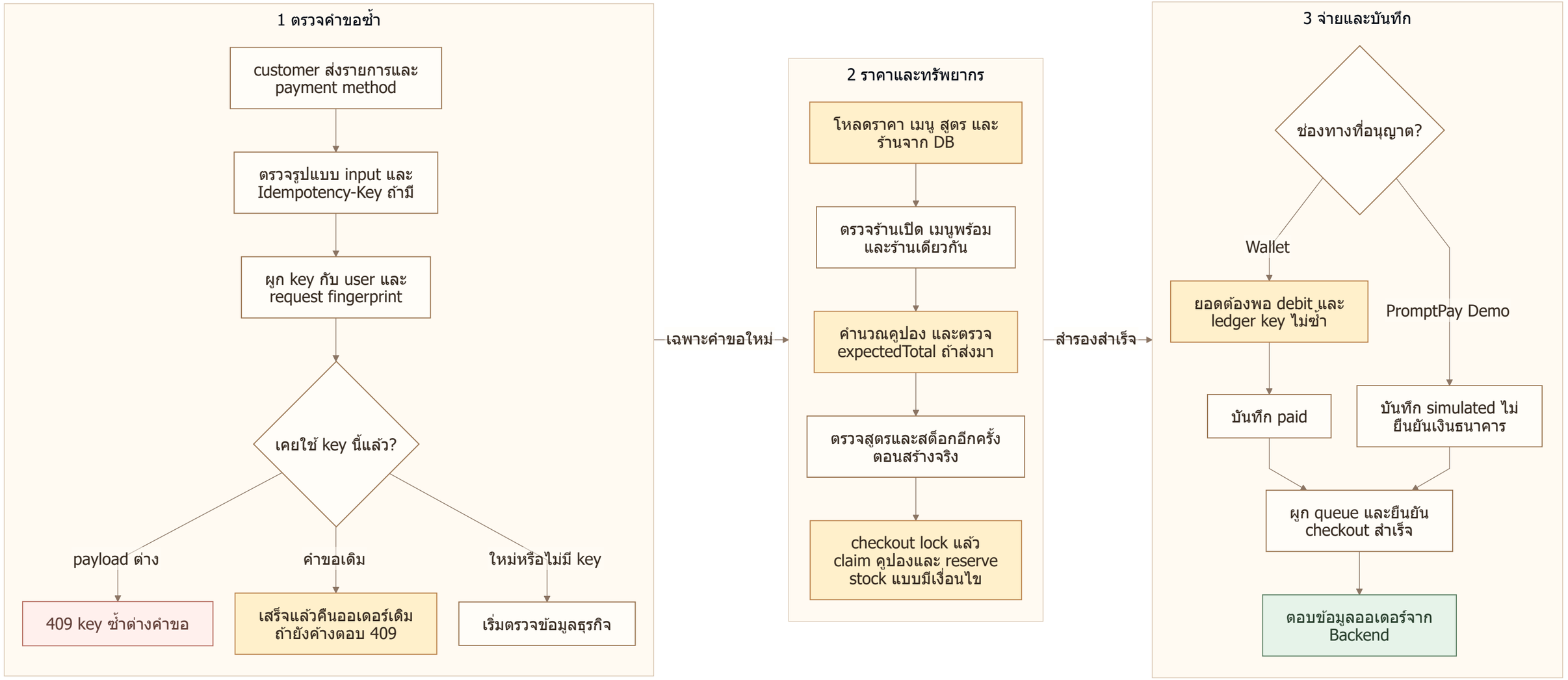
3. หลังเปลี่ยนอีเมลจะยกเลิกการผูก Google เดิมและลิงก์ reset เดิม ส่วน HTTP session เก่าไม่ผ่าน version check; Browser ที่ยืนยันได้รับ cookie ใหม่

4. ข้อจำกัด: flow ปัจจุบันไม่ได้ถาม current password หรือทำ re-auth ก่อนเริ่ม ใช้ session ที่มีอยู่และ OTP กล่องเมลใหม่ จึงควรเสริม step-up สำหรับการเปลี่ยนข้อมูลสำคัญ

Source: email-change.js · EmailChange.js · customerController.js · customerRoutes.js | รายละเอียดอ้างอิงใน README

08 สั่งซื้อ: เซิร์ฟเวอร์เป็นผู้ตัดสินราคา สต็อก และยอดเงิน

ไม่เชื่อราคา ยอดรวม หรือสถานะจ่ายเงินที่ Browser ส่งมา; ตรวจอีกครั้งเมื่อสร้างออเดอร์จริง



ข้อมูลอ้างอิง: DB ราคา/สูตร/คูปอง | ข้อมูลกันซ้ำ: key + fingerprint | เงิน: Wallet ledger | สต็อก: reservation

1. Frontend preflight ตรวจวัตถุดิบก่อนคิดเงินเพื่อแจ้งผู้ใช้ ส่วน createOrder ตรวจและสำรองซ้ำเพื่อรับมือสต็อกเปลี่ยนระหว่างกดสั่ง; สูตรผูกวัตถุดิบต้องพร้อมก่อนขาย

2. คูปองเลือกได้ 1 ใบต่อออเดอร์ ต้องเป็นสิทธิ์ของลูกค้าและผ่านร้าน วันเวลา ยอดขั้นต่ำ และโควตา; ราคาและตัวเลือกอ้างอิงเมนูในฐานข้อมูล

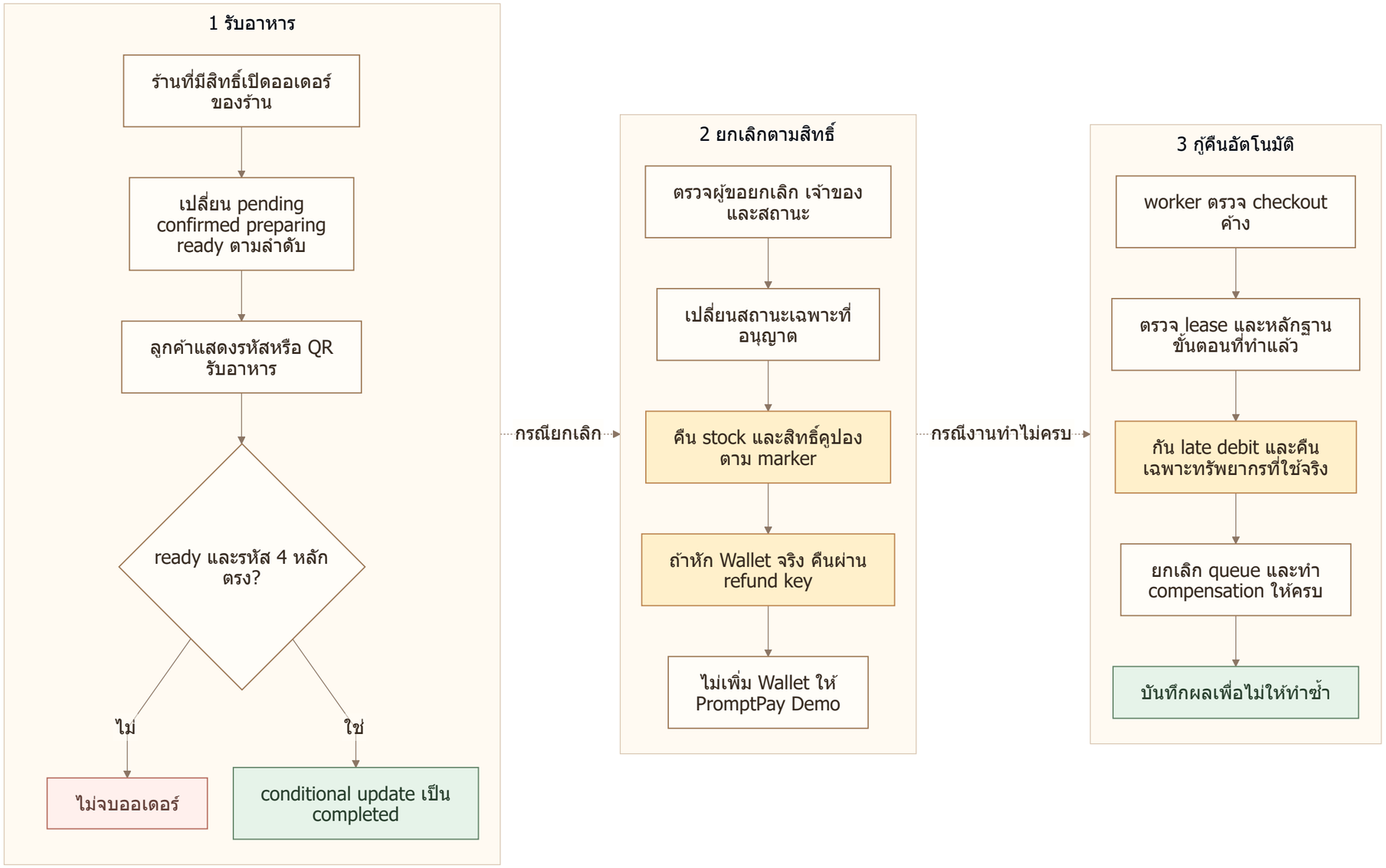
3. ถ้าส่ง Idempotency-Key จะผูกกับลูกค้าและ fingerprint คำขอ; header ยัง optional สำหรับ legacy clients จึงไม่รับประกันกันซ้ำทุก request ส่วน Wallet ใช้เงื่อนไขยอดเงินและ key ของ ledger

4. ใช้ atomic updates หลายจุด + checkout lock + compensation ไม่ใช่ transaction ครอบทุก collection; ค่าตั้งต้นเป็น Demo รองรับโหมด Wallet และไม่พบ payment gateway ธนาคารจริง; ครั้งนี้ไม่ตรวจ PAYMENT_MODE จริง

Source: orderController.js · inventory.js · order-pricing.js · checkout.js · payment-config.js · wallet.js · customer-coupons.js · Order.js | รายละเอียดอ้างอิงใน README

09 รับอาหาร ยกเลิก และกู้คืนงานที่ทำไม่ครบ

จำกัดการเปลี่ยนสถานะและใช้เงื่อนไขก่อนคืนทรัพยากร เพื่อลดการรับซ้ำ คืนซ้ำ และข้อมูลค้าง



pickup code: 4 หลัก | จบออเดอร์ต้อง ready + code ตรง | refund อ้างอิง debit จริง | recovery ไม่ใช่ DB backup

1. ร้านต้องผ่าน role และ store_id; receiveOrder ตรวจรูปแบบรหัส 4 หลัก ค่า code และสถานะ ready ก่อน compare-and-set เป็น completed จึงปิดงานซ้ำไม่ได้

2. ขอบเขตยกเลิก: ลูกค้า pending/confirmed, ร้านถึง preparing, admin ถึง ready โดยมีเงื่อนไข checkout; ไม่ใช่ทุกบทบาทยกเลิกได้ทุกสถานะ

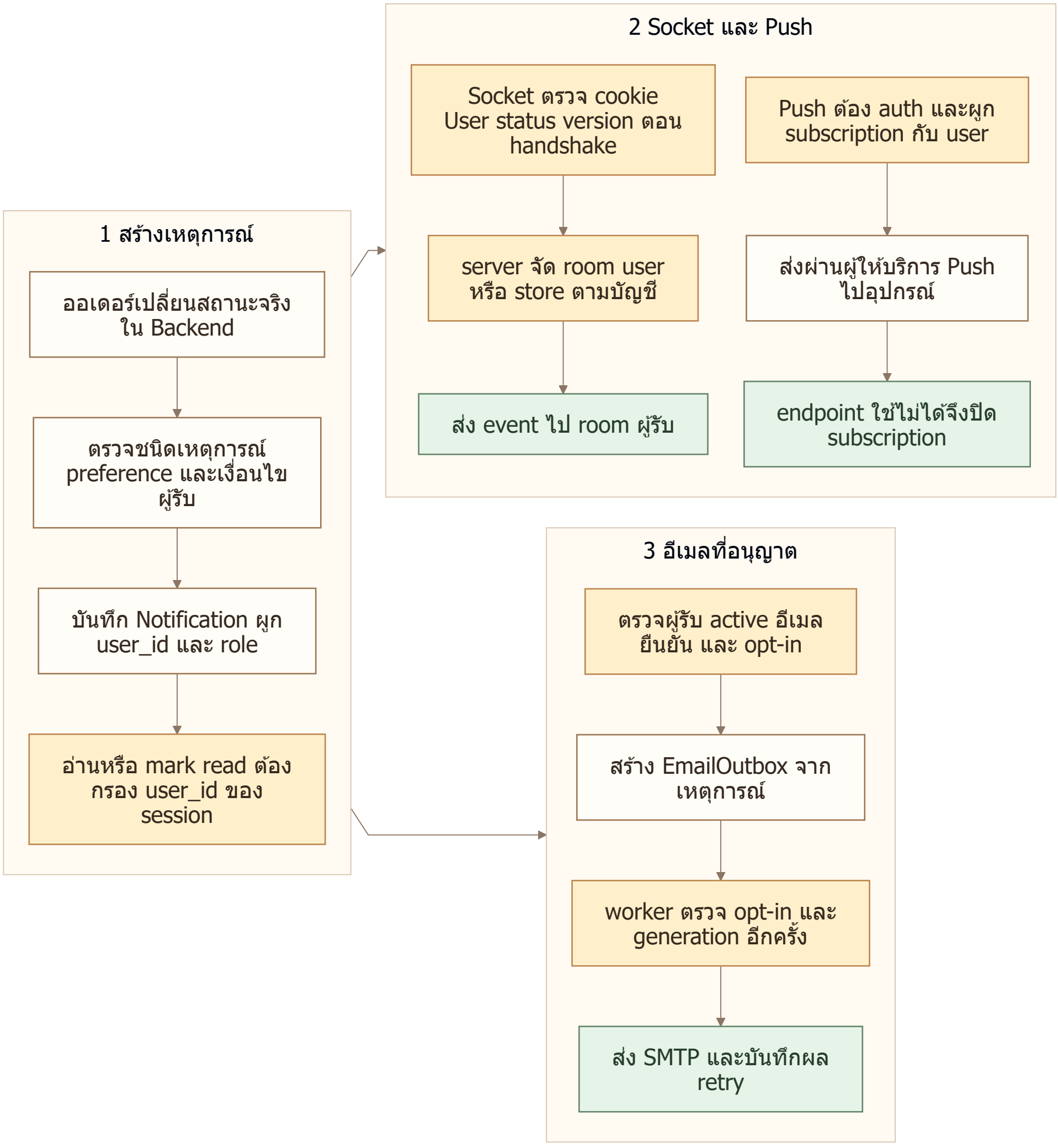
3. Recovery ทำตอนเริ่มและทุก 30 วินาที สำหรับ checkout ที่ค้างเกิน 2 นาทีและ lease ไม่ทำงาน คืนเฉพาะ debit/stock/coupon ที่มีหลักฐาน durable marker

4. ข้อจำกัดสำคัญ: pickup code อยู่ใน orderView ของร้านด้วย เป็นหลักฐานปฏิบัติงาน ไม่ใช่ความลับที่มีแต่ลูกค้าถือ; ไม่พบ attempt limit หรือ expiry เฉพาะรหัสรับอาหาร

Source: merchantController.js · orderController.js · adminController.js · order-lifecycle.js · checkout-recovery.js | รายละเอียดอ้างอิงใน README

10 แจ้งเตือน: ส่งให้บัญชีและอุปกรณ์ที่ถูกคน

Backend เป็นผู้เลือกผู้รับ ไม่ให้ Browser เลือก private room หรืออ่านรายการของบัญชีอื่นจาก ID อย่างเดียว



Notification ผูกผู้รับ | Subscription มี endpoint และ keys จึงต้องรักษาสิทธิ์ DB | อีเมลอาจถูกส่งไปผู้ให้บริการภายนอก

1. Socket.IO อนุญาต anonymous connection แต่ไม่เข้าห้อง private; server ตรวจ signed cookie + User + version ตอน handshake แล้วกำหนด room เอง

2. การแจ้งเตือนและ mark read กรองผู้ใช้; merchant notification routes รับเฉพาะเจ้าของร้าน role merchant แม้ routes ปฏิบัติงานร้านบางส่วนให้ staff/admin ใช้ได้

3. อีเมลออเดอร์ต้อง opt-in และ verified email; outbox ตรวจ generation อีกครั้งเพื่อไม่ส่ง backlog หลังปิด/เปิดใหม่ มี retry สูงสุด 5 ครั้ง จึงไม่รับประกันว่าจะถึงกล่องทุกครั้ง

4. Push endpoint จำกัด HTTPS และผู้ให้บริการที่อนุญาต; payload ไม่มี pickup code และ service worker เปิดลิงก์เฉพาะ origin เดิม Socket ตรวจสิทธิ์ตอน handshake ส่วนการส่งภายนอกยังมี crash window จึงไม่ใช่ exactly-once

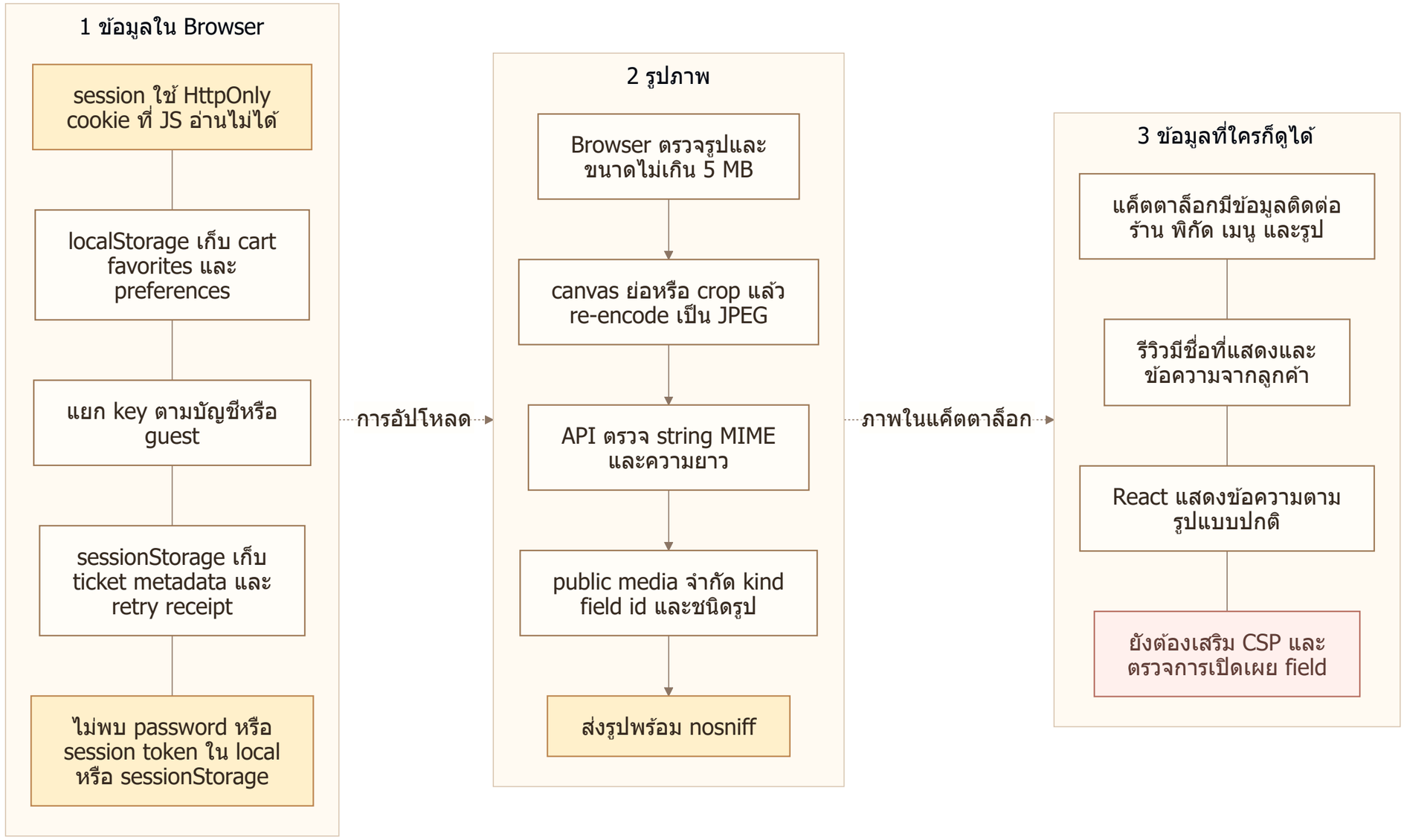
5. SMTP ขึ้นกับ config โดย Gmail บังคับ STARTTLS เมื่อไม่ใช้ implicit TLS แต่ไม่ได้บังคับ TLS ให้ทุกผู้ให้บริการ; Socket.IO ใช้ CORS config แต่ยังไม่มี allowRequest gate แยกสำหรับตรวจ WebSocket Origin

6. customer ที่ notifyReady=false จะไม่สร้างทั้ง inbox และ push ทุกสถานะ; merchant รับเฉพาะเหตุการณ์ที่กำหนด เช่น ออเดอร์ใหม่ที่จ่ายแล้ว ลูกค้ายกเลิก และรับอาหารเสร็จโดยมี watermark ร่วมด้วย

Source: server.js · notifications.js · order-email.js · mail.js · push-sw.js · PushSubscription.js · EmailOutbox.js · customerRoutes.js · merchantRoutes.js | รายละเอียดอ้างอิงใน README

11 Browser รูปภาพ และข้อมูลที่เปิดเผยต่อสาธารณะ

ป้องกันข้อมูลฝั่งหน้าเว็บด้วยการจำกัดการเก็บและการแสดงผล พร้อมแยกข้อมูลส่วนตัวออกจากข้อมูลที่ตั้งใจเผยแพร่



เส้นไม่มีหัวลูกศร = รายการในกลุ่ม | Browser storage ≠ ที่เก็บ secret | MIME ≠ ยืนยันเนื้อไฟล์ปลอดภัย

1. state ที่ persist ตัด user/orders/coupons ออก แต่ยังมี cart, favorites, notification summaries และ preferences จึงห้ามอ้างว่า Browser ไม่เก็บข้อมูลผู้ใช้เลย; เครื่องที่ใช้ร่วมกันยังต้องดูแลข้อมูลค้าง

2. ไม่พบ dangerouslySetInnerHTML หรือ eval ในการค้น frontend ปัจจุบัน ใช้ React text rendering ตามปกติ; นี่ไม่ใช่หลักฐานว่าป้องกัน XSS ได้ทุกทางหรือมี sanitizer ครบ

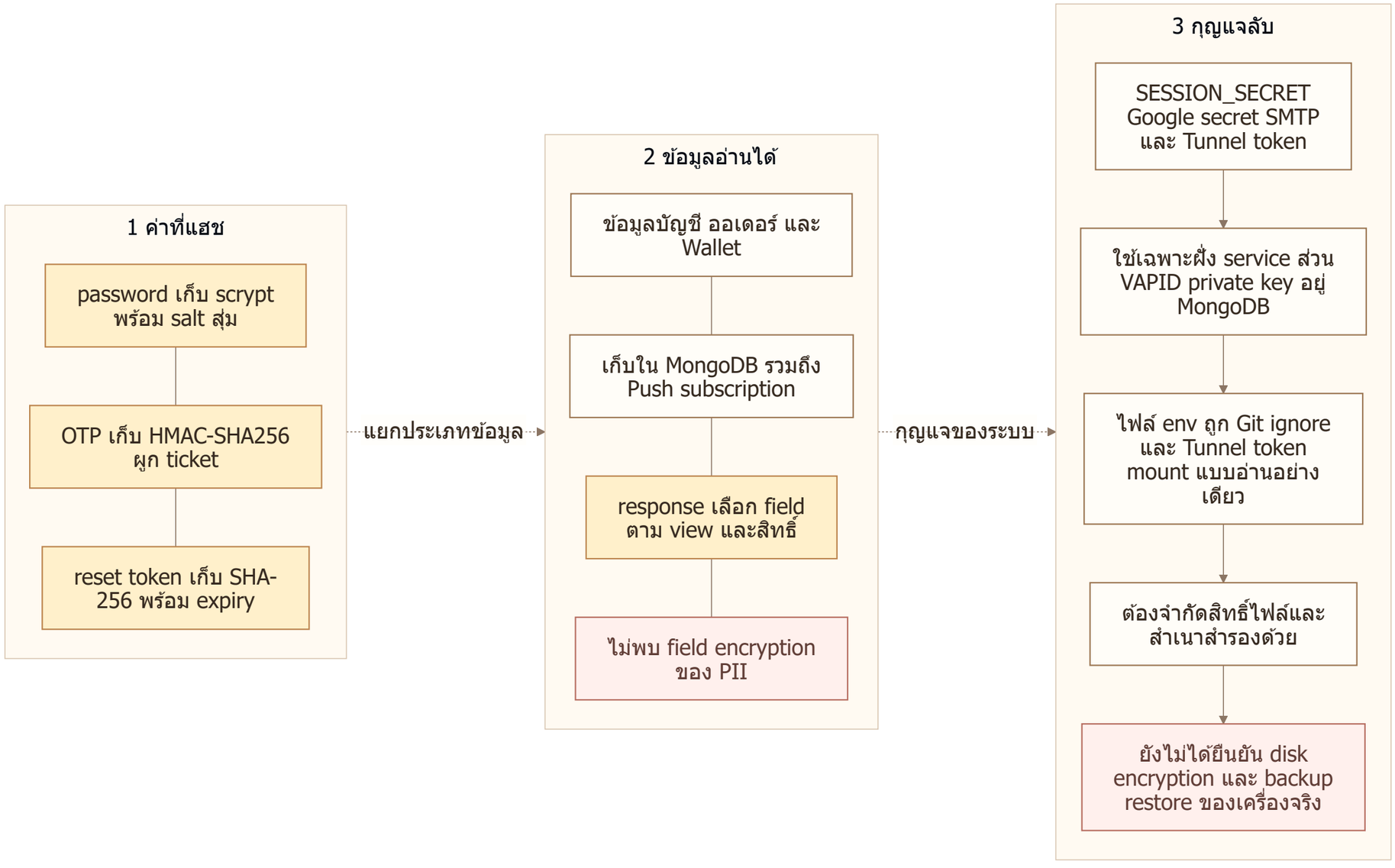
3. Server ตรวจรูปด้วยรูปแบบ string/MIME ไม่ได้ถอด bytes ตรวจ signature/dimension หรือสแกน malware; frontend validation ไม่แทน server validation และยังรับ URL รูปภายนอก HTTP(S)

4. ข้อมูล public มี store statusNote, สูตร ingredient ID/ปริมาณในเมนู และชื่อ/อ้างอิงออเดอร์ในรีวิว; ควรทบทวน data minimization ส่วน CSV export ยังไม่ neutralize formula prefixes

Source: customer-provider.jsx · store.js · checkout-attempt.js · image.js · menu-validation.js · media.js · publicController.js · csv.js | รายละเอียดอ้างอิงใน README

12 เก็บข้อมูลอะไร และส่วนใดเป็นความลับ

จำแนกข้อมูลที่แฮช ข้อมูลที่อ่านได้ และกุญแจฝั่งเซิร์ฟเวอร์ เพื่อไม่สับสนว่าทั้งฐานข้อมูลถูกเข้ารหัสแล้ว



Hash ใช้ตรวจความถูกต้อง | Signature ใช้ตรวจการแก้ไข | Encryption ใช้ปกปิดข้อมูล | Backup ใช้กู้คืน

1. User response แบบ userView ไม่คืน password_hash, reset_token_hash, session_version หรือ google_id; แต่การจำกัด response ไม่ได้เข้ารหัสข้อมูลในฐานข้อมูล

2. SESSION_SECRET ใช้กับลายเซ็น session/state และ HMAC บาง flow; ฟังก์ชันสร้าง/ตรวจ session ปฏิเสธค่าว่างหรือค่าตัวอย่างที่กำหนดเมื่อเป็น production ไม่ใช่ startup validation ครอบทุก flow; เปลี่ยน key กระทบ token เดิม

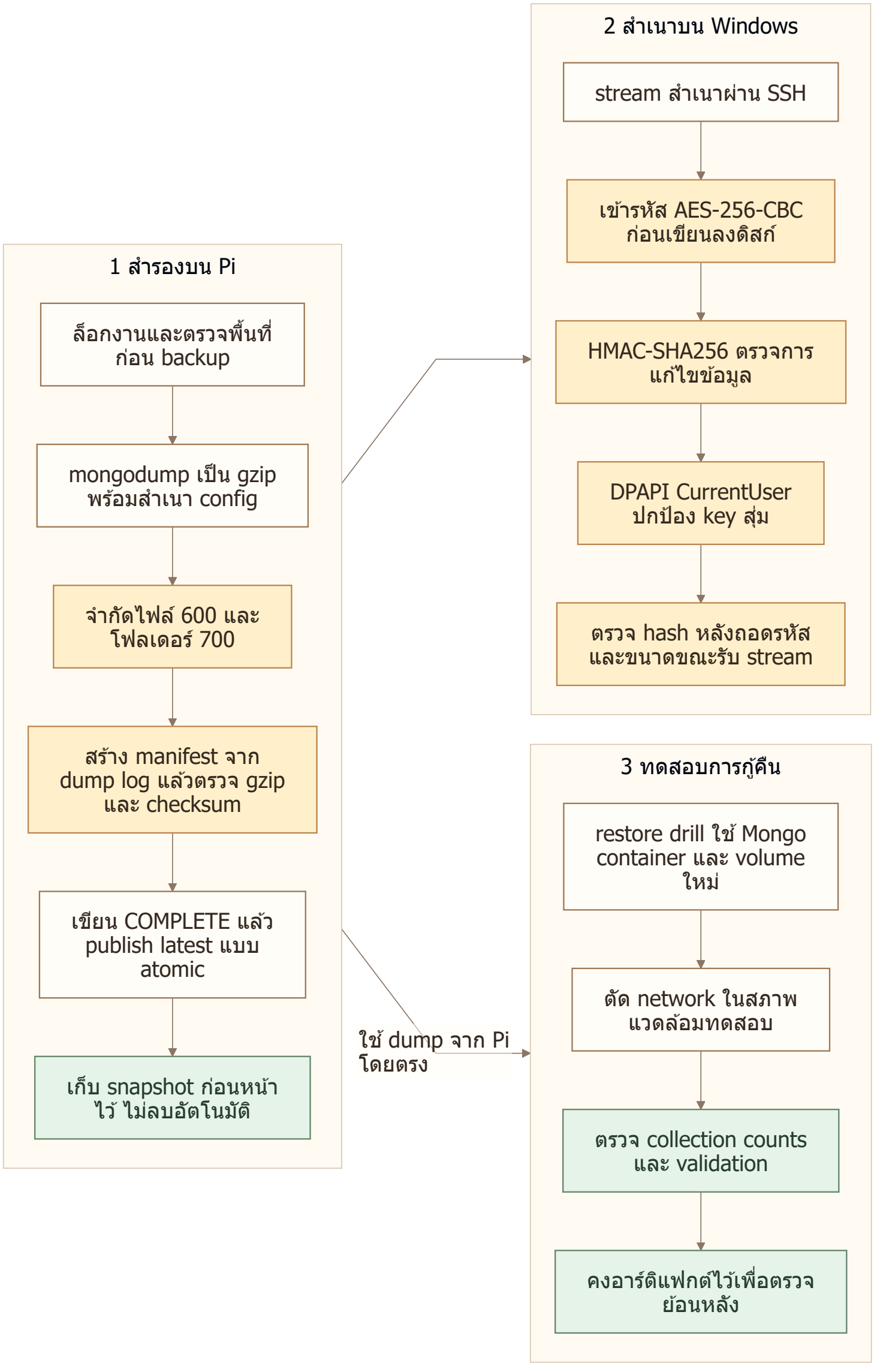
3. Git ignore ช่วยไม่ stage ไฟล์ลับตามปกติ แต่ไม่เข้ารหัสไฟล์ ไม่ถอน secret ที่เคย commit และไม่ป้องกันผู้ที่อ่านไฟล์บนเครื่องได้; ไม่คัดลอก secret จริงเข้าเอกสารนี้

4. การสำรองช่วยเรื่องกู้คืน ไม่ใช่การเข้ารหัสโดยตัวมันเอง ต้องจำกัดผู้เข้าถึง เข้ารหัสสำเนา และทดสอบ restore; การตรวจครั้งนี้เป็น source audit จึงไม่ยืนยันการตั้งค่าหรือ backup เครื่องจริง

Source: User.js · password.js · session.js · user-view.js · PushSubscription.js · PushConfig.js · .gitignore · .dockerignore · .env.example · tunnel.compose.yaml | รายละเอียดอ้างอิงใน README

13 สำรองและกู้คืน: รักษาทั้งความลับและความพร้อมใช้

ผังนี้อธิบายสคริปต์ที่มีใน repository เท่านั้น ยังไม่ยืนยันว่า schedule ล่าสุดทำงานหรือ restore สำเร็จบนเครื่องจริง



Pi: plaintext + permissions | SSH: เข้ารหัสระหว่างส่ง | Windows: AES + HMAC + DPAPI | Restore: ทดสอบแยกจากระบบจริง

1. บน Pi snapshot เป็นไฟล์ gzip และสำเนา config ที่ยังเป็น plaintext แม้สิทธิ์ owner-only จึงมีทั้งข้อมูลผู้ใช้และ secret ที่ต้องดูแล; gzip คือการบีบอัด ไม่ใช่ encryption

2. สำเนา Windows ใช้ encrypt-then-HMAC และ DPAPI ผูก key กับ Windows user ปัจจุบัน การกู้คืนจึงต้องมีสิทธิ์/กุญแจที่เกี่ยวข้อง; ไม่ได้อ้างว่า backup บน Pi ถูกเข้ารหัสด้วย

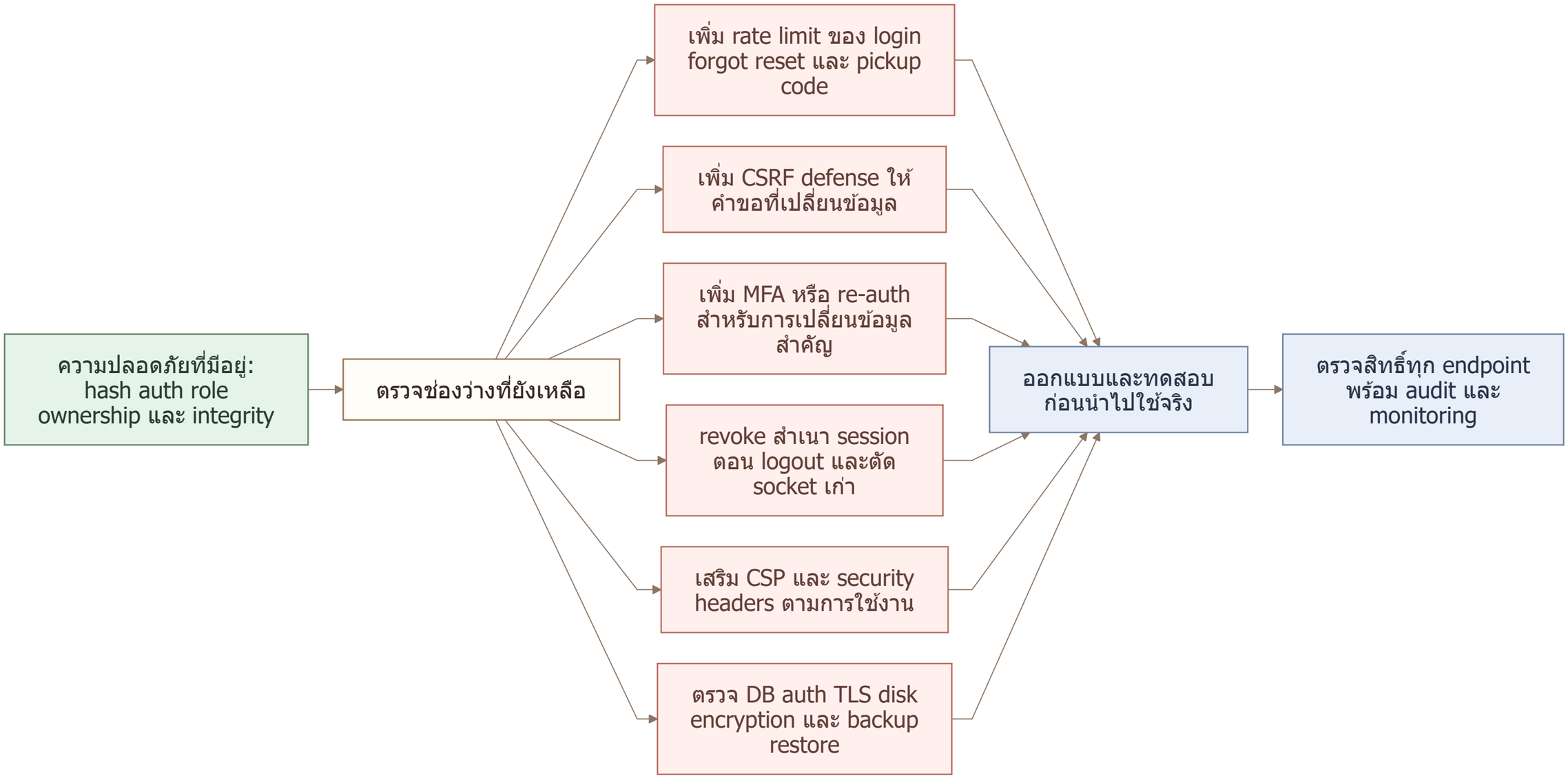
3. Online mongodump จาก standalone MongoDB เป็น collection-level copy ไม่ใช่ snapshot ที่ atomic ข้ามทุก collection; การตรวจ checksum ยืนยันไฟล์คงเดิม ไม่รับประกันความสอดคล้องธุรกิจทุกกรณี

4. สคริปต์มี daily task ที่ต้อง user sign in และตรวจ backup เก่าเกิน 36 ชั่วโมง แต่ครั้งนี้ไม่ได้อ่านรายงานจริง จึงไม่ระบุ RPO/RTO หรือรับประกันว่ามี backup/restore ล่าสุดสำเร็จ

Source: backup-pi.sh · backup-windows-crypto.ps1 · backup-sync-windows.ps1 · backup-install-windows.ps1 · backup-restore-drill.sh | รายละเอียดอ้างอิงใน README

14 สิ่งที่ยังควรเสริมก่อนยกระดับใช้งานจริง

หน้านี้เป็นข้อเสนอจากการอ่านโค้ด ไม่ใช่ฟังก์ชันที่เพิ่มแล้ว และไม่ใช่ผลทดสอบเจาะระบบ



ลำดับแนะนำ: ปิดช่อง abuse/CSRF → เพิ่ม re-auth/revocation → ตรวจ runtime/storage → ทดสอบสิทธิ์และเฝ้าระวัง

1. ควรเริ่มจาก rate limit ในช่องทางเสี่ยงและ CSRF defense: session บน HTTPS ใช้ SameSite=None; CORS กับ HttpOnly ช่วยคนละเรื่องและไม่ใช่ CSRF protection ที่ครบถ้วน

2. Logout ปัจจุบันลบ cookie เท่านั้น; เพิ่มการ revoke ที่เหมาะกับ per-session/device และตรวจสิทธิ์/อายุของ Socket.IO ต่อเนื่อง โดยระวังผลต่ออุปกรณ์ที่ใช้งานอยู่

3. ไม่พบ MFA, CSP/Helmet หรือ field encryption ในแอป; การเพิ่มต้องออกแบบให้ตรงความเสี่ยง อย่าอ้างว่าการมีไลบรารีหรือเครือข่ายภายในทำให้ปลอดภัยทุกกรณี

4. ตรวจ runtime แยกจาก source: Mongo authentication, firewall, TLS, secret permissions/rotation, patching, encrypted backup และ restore drill ยังไม่ยืนยันจากงานเอกสารนี้

Source: authRoutes.js · authController.js · session.js · merchantController.js · server.js · db.js · googleAuthController.js | รายละเอียดอ้างอิงใน README